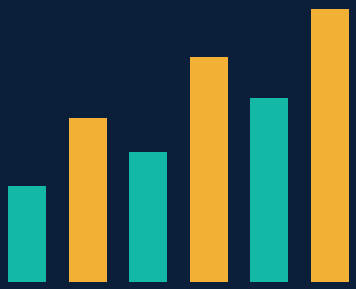

PYTHON • PANDAS • NUMPY

# STUDENT PERFORMANCE

## Project Documentation

**PROJECT**
Student Performance

**AUTHOR**
Marisha Bansal

**TECHNOLOGY STACK**
Python · Pandas · NumPy · CSV

**DEVELOPMENT ENVIRONMENT**
Visual Studio Code

**GITHUB REPOSITORY**
github.com/marishaa-08/Student-Performance

*A data-driven academic analysis tool designed to transform student records into clear, interpretable performance insights.*

PREPARED FOR ACADEMIC PROJECT SUBMISSION

## 1 Project Overview

The **Student Performance** project is a compact, data-oriented Python application that evaluates academic records and presents meaningful summaries of student achievement. Through structured data processing, it computes marks, averages, grades, result classifications, subject-level statistics, and selected learning-pattern indicators.

Rather than treating a spreadsheet as a static list of marks, the program transforms raw student records into an organised analytical report that supports rapid review and meaningful comparison.

## 2 Objective

The principal objective is to demonstrate how **Python, Pandas, and NumPy** can be applied to a practical academic dataset. The program aims to:

- Calculate each student's total and average marks across three subjects.
- Assign grades and determine pass/fail status according to the rules implemented in the program.
- Summarise class-wide performance and subject-wise mark distributions.
- Identify the three highest-performing students by average marks.
- Explore descriptive relationships between study hours, attendance, assignments, laboratory performance, and marks.
- Export the processed records for subsequent inspection or reuse.

## 3 Technologies and Tools

| Technology | Purpose in the Project |
|---|---|
| **Python** | Core language for program flow, functions, conditions, loops, and exception handling. |
| **Pandas** | Reads the CSV dataset into a DataFrame and supports data cleaning, calculations, sorting, and tabular summaries. |
| **NumPy** | Supports numerical operations and the calculation of a correlation coefficient. |
| **CSV** | Stores the source student records in a simple, portable tabular format. |
| **Visual Studio Code** | Provides the editor and terminal used to develop and run the application. |

## 4 Dataset Description

The source file, **students.csv**, contains **20 fictional student records**, each representing one student in the BCA department. The dataset is intended for educational demonstration rather than as a record of real students.

| Category | Fields |
|---|---|
| **Identification** | Student_ID, Name, Department |
| **Subject marks** | Python, Mathematics, Computer_Fundamentals |
| **Learning & engagement indicators** | Attendance, Weekly_Study_Hours, Assignments_Completed, Lab_Performance |

Subject marks and laboratory performance are expressed on a 0–100 scale; attendance is a percentage; assignments completed are recorded out of 10. The program's validation logic verifies the expected columns and selected value ranges before proceeding.

## 5 Implementation Workflow

| | |
|---|---|
| 1 | **Data Ingestion and Validation**<br>Pandas reads the CSV file into a DataFrame. The program confirms that all required columns exist and validates relevant values before analysis begins. |
| 2 | **Performance Computation**<br>Total_Marks is derived by summing the three subject marks, and Average_Marks by dividing that total by the number of subjects. |
| 3 | **Grade and Result Classification**<br>A grade is assigned according to the thresholds defined in the source code, while pass/fail status is determined by the program's subject-mark condition. |
| 4 | **Class and Subject Summaries**<br>The application reports the number of students, class average, highest and lowest student averages, pass/fail counts, and subject-wise averages and extremes. |
| 5 | **Top-Performer Identification**<br>Records are sorted by average marks so that the three highest averages can be displayed. |
| 6 | **Learning-Pattern Exploration**<br>The program summarises study hours, attendance, assignment completion, and lab performance, then computes the correlation between weekly study hours and average marks. |
| 7 | **Practical Comparison and Export**<br>A practical-performance gap is calculated by comparing lab performance with average subject marks. The processed results are written to student_analysis_results.csv. |

## 6 Principal Features

- CSV-based dataset ingestion and validation.
- Automated computation of total marks, average marks, grades, and pass/fail outcomes.
- Class-level performance summary and subject-wise analysis.
- Ranking and display of the top three students.
- Descriptive analysis of study habits and engagement indicators.
- Theory-average versus laboratory-performance comparison.
- Export of analysed records to a new CSV file.

## 7 Sample Execution Results

In the successful terminal run recorded during development, the program produced the following class-level summary:

| Metric | Observed Output |
|---|---|
| Total students | **20** |
| Class average | **78.37** |
| Highest student average | **95.67** |
| Lowest student average | **47.33** |
| Students passed | **19** |
| Students failed | **1** |

The same run reported a correlation of approximately **0.77** between weekly study hours and average marks.

**INTERPRETATION** This indicates a positive association within a small, fictional dataset; it does not establish that additional study hours cause higher marks. Results should be re-verified whenever the code or dataset is modified.

**OUTPUT EVIDENCE**
Insert screenshots from your own successful run: the student table, class summary, subject-wise analysis, top-three students, learning-pattern analysis, and practical comparison. Screenshots are intentionally not fabricated in this document.

## 8 Outcome and Learning

The project demonstrates a complete introductory data-analysis workflow: reading structured records, validating data, deriving new columns, summarising groups, sorting results, applying conditional logic, and exporting an analytical dataset.

Key learning areas during development include working with Pandas DataFrames, applying NumPy to numerical analysis, decomposing a program into reusable functions, handling common data errors, and communicating results through readable terminal output.

## 9 Challenges and Considerations

- Ensuring that the CSV file resides in the same project folder as **main.py** so that the program can locate it.
- Keeping column names in **students.csv** consistent with those expected by the Python code.
- Validating mark ranges and other numeric fields before performing calculations.
- Interpreting correlation carefully and avoiding unsupported causal conclusions.

## 10 Project Files and Execution

The repository's core files are:

| File | Description |
|---|---|
| **main.py** | The Python source code. |
| **students.csv** | The fictional input dataset. |
| **README.md** | Repository overview and basic usage instructions. |
| **Student_Performance_Project_Documentation.pdf** | This project report. |

To run the project from the VS Code terminal, ensure Pandas and NumPy are installed, open the project folder, and execute:

```
$ python main.py
```

## 11 GitHub Repository

The source code and dataset are published in the following repository:

https://github.com/marishaa-08/Student-Performance

> **CHECKLIST** Before final submission, confirm that the repository contains the latest **main.py**, **students.csv**, **README.md**, and this PDF documentation. If you add or change files locally, commit and push the updates to GitHub.

## 12 Conclusion

*Student Performance* illustrates how foundational Python programming can be combined with Pandas and NumPy to produce a concise, reproducible academic-performance report. The application converts a small dataset into structured summaries and an exportable results file, while providing practical experience in data validation, descriptive statistics, and program organisation.